

ARM-Sparse: A Reproducible Investigation of the Quality–Latency Break-Even for Dynamic FFN Block Sparsity on Apple M2

Sanjay Elango*

Chennai Institute of Technology, Chennai, India
sanjayelango06@gmail.com

Technical report, September 2026

Abstract

Dynamic feed-forward-network (FFN) sparsity skips neurons that contribute little to the current token, but on CPUs the irregular memory access of neuron-level masks can erase the expected speedup. A common compromise is to skip fixed contiguous *blocks* of neurons. We ask whether block sparsity can both preserve model quality and beat an optimized dense baseline on an Apple M2 CPU, measuring the two separately: how much oracle-selected block sparsity a model tolerates under a pre-specified 5% relative-perplexity gate, and how fast the resulting masks execute against Apple Accelerate. Blocks of eight neurons (B8) passed the gate only up to 10% sparsity for Llama 3.2 1B and 20% for TinyLlama 1.1B. A packed B8 executor was 27–28% slower than dense at the passing Llama point and first won only at 40–50% sparsity, so it has no quality–speed intersection. The same block-aligned masks run through a simple per-neuron executor with neuron-major weights, however, were faster than dense at the quality ceiling: by 0.9–3.5% on real Llama masks and 2.6–7.3% (Llama) and 5.1–7.3% (TinyLlama) on synthetic ones. Block alignment therefore helps, but through memory layout rather than packed block kernels, and the quality ceiling caps the achievable gain at 11% and 25% of FFN time even for a perfect kernel. Thirteen supporting experiments on layouts, SIMD, coalescing, and quantized paths, together with all raw evidence, are released for reproduction.

1 Introduction

Batch-one token generation on laptops and phones is dominated by reading weights. In decoder-only transformers the feed-forward network (FFN) holds roughly 80% of each layer’s parameters in the models we study, so reading fewer FFN weights per token is an attractive target. Contextual sparsity work observes that, for any given input, many FFN neurons contribute little to the output and could in principle be skipped [11, 9]. Recent training-

free methods report that large fractions of SwiGLU [16] activations can be zeroed with modest quality loss [8, 10].

A FLOP count is not a CPU performance model, however. Selecting individual neurons produces scattered weight reads and per-neuron dispatch that compete with cache lines, hardware prefetchers, and vector (SIMD) units, whereas a dense matrix–vector product (GEMV) streams weights sequentially through a heavily tuned vendor library. A common response is to impose structure: partition the intermediate dimension into fixed contiguous blocks of B neurons and skip whole blocks, computing some unnecessary neurons in exchange for regular memory access. Clustered execution units already appear in storage- and scheduler-oriented systems [1, 20]. What is less often reported is whether a block granularity can satisfy a quality constraint and a strong dense latency baseline *at the same time*, on the same hardware.

We answer that question for one constrained setting: a single-threaded Apple M2 laptop CPU running the FFNs of Llama 3.2 1B and TinyLlama 1.1B. The design deliberately favors sparsity—oracle masks, no selector cost, and FFN-only timing—so that a failure is informative. Our main findings are that (i) block structure costs far more quality than sparsity itself; (ii) a packed block executor never reaches a quality-compatible speedup; but (iii) the same block-aligned masks executed by a simple per-neuron kernel with neuron-major weights do beat dense, by a few percent, at the quality ceiling.

Contributions.

- A separated protocol that measures block-sparse *quality* (oracle masks, pre-specified perplexity gate) and *latency* (replayed masks, vendor-BLAS dense baseline) independently, then intersects them.
- A *structure tax* measurement: at 30% sparsity, neuron-level oracle masks cost 0.2–0.3% perplexity while contiguous B8 blocks cost 6.9–20.2%, and naively expanding neuron masks to blocks removes virtually all sparsity.
- A two-executor latency frontier on identical masks: packed B8 first beats dense at 40–50% sparsity, beyond the 10–20% quality ceiling, while a per-neuron

*Code, raw measurements, validators, and manuscript sources:
<https://github.com/sanjay0612/sparsityARM>

executor is faster than dense at every measured block-aligned sparsity, including the quality ceiling.

- A first-order bandwidth model showing that, at the quality ceiling, even a perfect kernel could save at most 11% (Llama) or 25% (TinyLlama) of FFN time, which bounds the value of any block-sparse kernel in this regime.
- Thirteen supporting experiments and a reproducible artifact with raw timings, hashes, and independent validators.

We do not claim novelty for contextual sparsity, neuron grouping, or weight packing. The contribution is a carefully bounded, reproducible measurement of where block sparsity does and does not pay off on one ARM laptop CPU.

2 Related Work

Activation and contextual sparsity. Li et al. observed that trained ReLU transformers naturally exhibit sparse FFN activations [9], and MoEfication showed that FFN neurons can be grouped into expert-like clusters after training [22]. DeJaVu introduced lightweight input-dependent predictors for contextual sparsity in MLP and attention blocks and demonstrated GPU speedups [11]. Because modern models use smooth SwiGLU activations [16] rather than ReLU, later work either restores ReLU through fine-tuning [13] or thresholds activations without training [8, 10]. These methods operate at neuron or channel granularity; ARM-Sparse asks what happens to quality when that granularity is coarsened into contiguous blocks for CPU efficiency.

Sparsity-aware systems for constrained devices. LLM in a Flash showed that loading bundled rows and columns in larger contiguous chunks improves flash throughput [1], which motivates our layout hypothesis, although our weights stay in RAM and our objective is CPU latency. PowerInfer exploits hot/cold neuron locality to split work between consumer CPUs and GPUs [17], and PowerInfer-2 elevates neuron clusters to scheduling and storage units on smartphones [20]. Dynamic Input Pruning jointly studies SwiGLU pruning and cache-aware masking on resource-constrained hardware [2]. Clustered execution is therefore prior art; our study differs by fixing hardware, mask budgets, and a strong dense comparator and measuring exactly where block execution crosses both a quality gate and dense latency.

Static and structured weight sparsity. Post-training pruning methods such as SparseGPT [3] and Wanda [18] remove weights permanently, and hardware-supported 2:4 sparsity [14] and block-sparse GPU kernels [6] show that structure is usually required before sparsity becomes speed. A broad survey is given by Hoeft et

al. [7]. ARM-Sparse is dynamic (masks change per token) and targets a CPU without sparse tensor hardware, which makes the structure–quality trade-off central.

Performance modelling. Our break-even analysis follows the spirit of the roofline model [19]: at batch size one, FFN execution is a sequence of matrix–vector products whose cost is governed mainly by how efficiently weights are streamed from memory.

3 Method

3.1 Block-sparse SwiGLU FFN

For a token representation $x \in \mathbb{R}^h$, a SwiGLU FFN with intermediate width m computes

$$y = W_d a, \quad a = \text{SiLU}(W_g x) \odot W_u x, \quad (1)$$

with $W_g, W_u \in \mathbb{R}^{m \times h}$ and $W_d \in \mathbb{R}^{h \times m}$. We partition the m intermediate neurons into $M = m/B$ groups of B adjacent neurons. For block b with activations $a_b \in \mathbb{R}^B$ and down-projection columns $W_{d,b} \in \mathbb{R}^{h \times B}$, the FFN output is $y = \sum_b W_{d,b} a_b$. A block-sparse FFN with active set $\mathcal{A}$ computes only $\hat{y} = \sum_{b \in \mathcal{A}} W_{d,b} a_b$. Block sparsity s is the fraction of blocks skipped, so $|\mathcal{A}| = (1 - s)M$. We write B8 for $B = 8$ and, for example, B8/20% for B8 at 20% block sparsity. We tested $B \in \{8, 16, 32, 64\}$ during the investigation; the frontier uses B8 because it preserved the most quality.

3.2 Oracle block selection

To measure whether a sparsity budget is *attainable* at all, we select blocks with an oracle that observes the dense post-SwiGLU activations of the current token and layer. An initial screen (EXP-003) ranked blocks by summed absolute activation; all frontier results use the output contribution norm

$$\text{score}(b) = \|W_{d,b} a_b\|_2^2 = a_b^\top (W_{d,b}^\top W_{d,b}) a_b, \quad (2)$$

computed exactly from a precomputed $B \times B$ Gram matrix per block. The top-scoring $(1 - s)M$ blocks are kept independently for every token and every FFN layer. The score ignores interactions between blocks, so it is a strong reference rather than the optimal subset, which is combinatorial. Because it reads information unavailable before the FFN runs, it is a favorable reference for what a deployable predictor could achieve, not a predictor. Neuron-level comparisons use the analogous top- k over individual absolute activations.

3.3 Executors

All three executors compute the same SwiGLU FFN and differ only in which weights they read and how those weights are laid out.

Dense baseline. Three single-precision matrix–vector products via Apple Accelerate (`cblas_sgemv`) with a SwiGLU loop between them. We chose Apple’s tuned vendor BLAS to avoid a weak comparator; we did not benchmark other dense paths such as BNNS. Accelerate may internally use Apple’s undocumented matrix coprocessor; we treat it as a black box.

Packed B8 executor. Gate and up rows of a block are contiguous in row-major storage; down-projection columns are repacked offline into a $[M, h, B]$ tile layout so that each block’s down weights are contiguous. For each active block the executor computes B gate and up dot products, applies SwiGLU, and accumulates the $h \times B$ down tile into the output as h dot products of length B (interleaved execution).

Per-neuron executor. Down weights are stored neuron-major, $[m, h]$, so each neuron’s gate, up, and down weights are each one contiguous row of h values. For each active neuron the executor computes two length- h dot products, applies SwiGLU, and adds a length- h scaled row to the output. Given a block mask it performs exactly the same arithmetic as the packed B8 executor; on B8-aligned masks, neuron-major storage is equivalent to a $[M, B, h]$ block layout.

Custom kernels are C++ compiled with `clang++ -O3` and rely on compiler auto-vectorization (NEON); allocation and packing are excluded from timed regions.

3.4 Selector–executor separation

Quality and latency are measured on separate paths. Quality experiments apply oracle masks inside the full model and report perplexity. Latency experiments replay fixed masks through isolated FFN executors. Selector cost is therefore excluded, which biases the study *toward* sparsity, and we make no end-to-end generation claim.

3.5 Acceptance rule

Before any quality experiment ran, we fixed a screening gate: a sparse condition is acceptable if its perplexity is at most 5% above the same-run dense perplexity. Grids and gates were committed to version control before results were recorded (e.g. the EXP-004 grid was locked in commit 849df02 before results commit 5b41874); there was no external registry. Runtime speedup is $100(t_{\text{dense}}/t_{\text{sparse}} - 1)$, so positive values mean the sparse executor is faster. A configuration has a *verified intersection* if one measured sparsity point both passes the quality gate and yields a positive speedup in every repeated run.

3.6 Provenance

Each result bundle records model revision, source commit, build command, operating system, data type, thread

count, seeds, masks, warm-ups, repetitions, raw retained timings, and aggregation. Independent validators recheck hashes, numerical tolerances, condition coverage, and recomputed medians. Figures 1 and 2 and Tables 1 and 2 are generated by a script from validated manifests; other numbers in the text were transcribed from the per-experiment findings and checked against them.

4 Experimental Setup

Hardware and software. All experiments ran CPU-only on an Apple M2 MacBook with 8 GiB of unified memory (Apple specifies 100 GB/s peak memory bandwidth for M2) under macOS 26.5.1, using one execution thread. No GPU, Metal, or Neural Engine path was used. Automated CPU-name capture failed under sandboxing, so the chip is operator-reported; llama.cpp independently detected Apple M2 in EXP-014. Accelerate was forced single-threaded (`BLASSetThreading` and `VECLIB_MAXIMUM_THREADS=1`).

Models. Llama 3.2 1B [5] (16 layers, $h = 2048$, $m = 8192$) at a pinned official revision, and TinyLlama 1.1B [21] (22 layers, $h = 2048$, $m = 5632$). Both use SwiGLU FFNs.

Quality protocol. We evaluate teacher-forced causal perplexity (PPL) on 16 deterministic WikiText-2 [12] test excerpts (the first 16 non-heading rows with at least 128 tokens, truncated to 128 tokens), giving 2,032 predicted tokens per condition. Inference uses BF16; masks, not arithmetic precision, are the variable under study, and we did not test whether the quality ceilings change in FP32. Masks are applied in every FFN layer and recomputed for every token. Per-record negative log-likelihoods are retained so aggregates can be independently recomputed. Dense perplexity was 22.774 for Llama, reproduced exactly across four experiments, and 18.017 for TinyLlama (one run).

Latency protocol. Runtime experiments time one single-token SwiGLU FFN with each model’s geometry ($2048 \rightarrow 8192 \rightarrow 2048$ and $2048 \rightarrow 5632 \rightarrow 2048$) using random FP32 weights, matching the single-precision Accelerate baseline. Each FFN call is timed with `std::chrono::steady_clock`. Executor order is randomly permuted in every iteration so that drift affects all executors equally. Each break-even case cycles eight masks, uses 20 warm-up iterations and 200 retained timings per executor, and is repeated with three independent weight/input seeds; we report per-run medians and the range across runs. Break-even masks select $(1 - s)M$ whole B8 blocks uniformly at random, so both sparse executors perform identical work; they are block-aligned but not otherwise clustered, and are not selector outputs. The quality-compatible Llama condition (B8/10%) is additionally replayed with 16 real masks recorded from one prompt

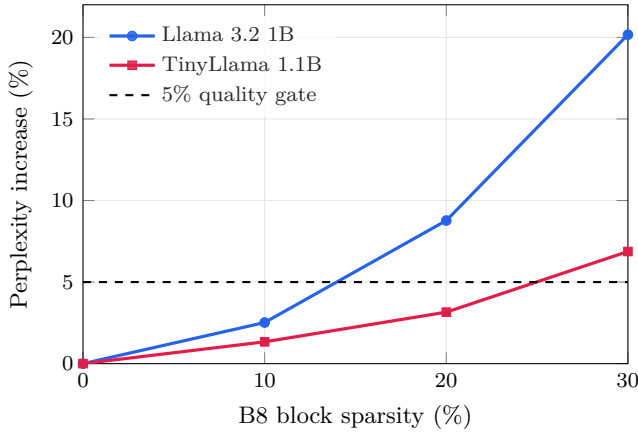


Figure 1: Relative perplexity increase for output-norm B8 oracle masks on 16 WikiText-2 excerpts. The dashed line is the pre-specified 5% gate.

token across all 16 layers.

Decisive experiments. Five experiments carry the conclusion: EXP-004 (Llama quality frontier), EXP-005 (real-mask replay of the passing Llama point), EXP-012 (Llama-geometry break-even), EXP-016 (TinyLlama quality frontier, with B32 as a granularity control), and EXP-018 (TinyLlama-geometry break-even). Thirteen further experiments are summarized in Section 6.4.

Bias toward sparsity. Oracle masks, zero selector cost, and FFN-only timing all favor the sparse method. A negative result under these conditions is strong evidence against the tested configuration, and a positive result is an upper bound on what a deployable system could achieve.

5 Results

5.1 Quality frontier

Figure 1 and Table 1 show the quality frontier. For Llama 3.2 1B, B8/10% increased perplexity by 2.52% and passed; 20% (+8.77%) and 30% (+20.17%) failed. TinyLlama was more tolerant, passing at 10% (+1.34%) and 20% (+3.16%) before failing at 30% (+6.87%). Coarser blocks were worse: Llama B16 missed the gate at 10% (+5.16%), and TinyLlama B32 passed only at 10% (+3.62%), failing at 20% (+10.31%). Because the degradation grows monotonically, we did not measure quality at 40% and beyond.

The structure tax. The quality loss is attributable mainly to contiguity rather than to sparsity itself. At the same 30% budget, neuron-level top- k oracle masks changed perplexity by only +0.21% (Llama, EXP-003) and +0.29% (TinyLlama, EXP-016), whereas B8 blocks cost +20.17% and +6.87%. Both models contain ample *unstructured* contextual sparsity; forcing it into groups of

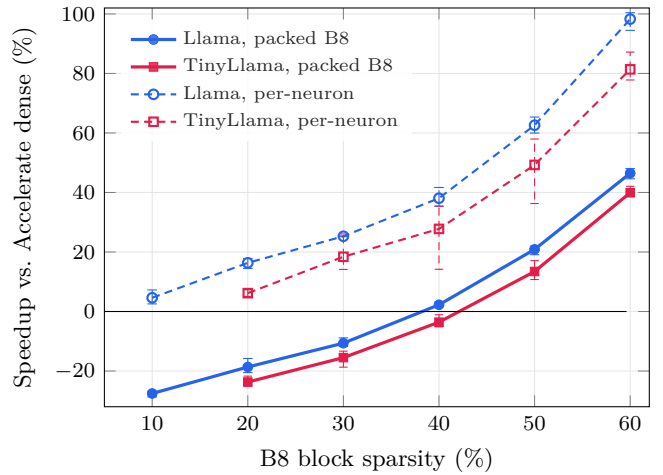


Figure 2: FFN speedup over Accelerate dense on identical block-aligned masks. Solid: packed B8 executor. Dashed: per-neuron executor with neuron-major weights. Points are means of three per-run medians; bars show the minimum and maximum run.

eight adjacent neurons destroys most of it. In the EXP-003 screen, random B8 masks at 30% raised Llama perplexity by 1,536% versus +21.3% for activation-ranked blocks, so block scoring carries real information—it is simply not enough.

5.2 Runtime frontier

Packed B8. For the Llama geometry, packed B8 was 8.9–11.7% slower than dense at 30% sparsity and first became faster at the 40% grid point, by 1.6–3.1% (Figure 2). For the TinyLlama geometry it was still 1.1–5.2% slower at 40% and first won at 50%, by 10.7–17.1%. The exact crossovers lie between grid points and were not interpolated. We did not isolate why the smaller FFN needs more sparsity to break even.

Per-neuron on the same masks. The per-neuron executor was faster than dense at every measured block-aligned sparsity, in every run. At the quality ceilings it gained 2.6–7.3% (Llama, 10%) and 5.1–7.3% (TinyLlama, 20%); at 30% it gained about 25% and 18% (Table 1). Since both sparse executors do identical arithmetic, the roughly 30-point difference between them at low sparsity comes from memory layout and loop structure (Section 6).

Real masks. At the quality-compatible Llama condition (B8/10%), we replayed real recorded masks (7,376 of 8,192 neurons active; 922 active blocks in a median of 93 contiguous runs). Dense took 4.01–4.13 ms per FFN. The per-neuron executor took 3.93–4.10 ms, a speedup of 0.9–3.5% in every run. Packed B8 took 5.54–5.74 ms, 37–39% longer than dense (a speedup of −27.1% to −27.9%), agreeing closely with the synthetic masks at the same sparsity (−26.8% to −28.7%).

Table 1: Full measured B8 grid. Δ PPL is relative perplexity increase over dense (%; $\checkmark$ passes the 5% gate). Speedups (%) are the mean of three per-run medians with [min, max] across runs; negative values mean the sparse executor is slower. Both executors run identical block-aligned masks with random FP32 weights in each model’s FFN shape (EXP-012, EXP-018); the real-mask Llama replay is reported in Section 5.2. “–” marks conditions that were not measured.

Sparsity	Llama 3.2 1B			TinyLlama 1.1B		
	Δ PPL	Packed B8	Per-neuron	Δ PPL	Packed B8	Per-neuron
10%	+2.52 $\checkmark$	−27.5 [−28.7, −26.8]	+4.6 [+2.6, +7.3]	+1.34 $\checkmark$	–	–
20%	+8.77 $\times$	−18.6 [−20.5, −15.8]	+16.3 [+14.5, +17.8]	+3.16 $\checkmark$	−23.7 [−25.0, −21.7]	+6.2 [+5.1, +7.3]
30%	+20.17 $\times$	−10.6 [−11.7, −8.9]	+25.3 [+25.1, +25.4]	+6.87 $\times$	−15.5 [−18.7, −13.3]	+18.4 [+14.1, +26.1]
40%	–	+2.3 [+1.6, +3.1]	+38.1 [+35.4, +41.7]	–	−3.6 [−5.2, −1.1]	+27.8 [+14.2, +35.4]
50%	–	+20.9 [+19.1, +22.0]	+62.6 [+60.0, +65.4]	–	+13.4 [+10.7, +17.1]	+49.3 [+36.3, +58.0]
60%	–	+46.5 [+44.6, +48.0]	+98.3 [+94.5, +100.4]	–	+39.9 [+38.8, +42.1]	+81.4 [+77.8, +87.2]
70%	–	+91.2 [+89.3, +93.6]	+160.9 [+157.7, +163.7]	–	–	–
80%	–	+185.5 [+178.6, +193.3]	+277.0 [+269.3, +284.7]	–	–	–

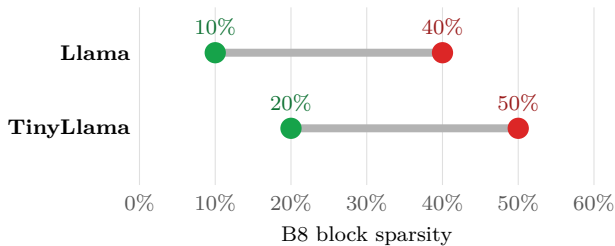


Figure 3: Highest B8 sparsity passing the quality gate (green) versus the first measured sparsity at which the *packed* B8 executor is reproducibly faster than dense (red).

Table 2: B8 quality–speed frontier. The last column is the per-neuron executor’s speedup range at the quality ceiling (synthetic block-aligned masks).

Model	Quality ceiling	First B8 speedup	Per-neuron at ceiling
Llama 3.2 1B	10%	40%	+2.6 to +7.3%
TinyLlama 1.1B	20%	50%	+5.1 to +7.3%

Alignment matters for the per-neuron executor too. In an earlier synthetic sweep (EXP-001) with the same per-neuron kernel, masks that selected individual neurons uniformly at random were slower than dense in 10 of 12 configurations at 20–40% sparsity (e.g. 5.28 versus 4.36 ms median at 30%). The per-neuron speedups above therefore depend on active neurons arriving in aligned runs, which is exactly what block masks provide.

5.3 Quality–speed intersection

For the packed B8 executor the intersection is empty (Figure 3, Table 2): its first measured speedup lies three grid points (30 percentage points) beyond the highest passing point for both models. Because the grid spacing is 10 points, the true gap is at least 10 points. For the per-neuron executor the intersection is *non-empty but small*:

B8/10% for Llama and B8/20% for TinyLlama pass the quality gate and are faster than dense in every run, by 0.9–3.5% on real Llama masks and 2.6–7.3% and 5.1–7.3% on synthetic masks.

6 Why the Gains Are Small

6.1 A first-order bandwidth model

At batch size one each FFN weight is used once per token, so latency is governed mainly by how quickly weights are streamed. The FP32 Llama FFN holds $3 \cdot 2048 \cdot 8192 \cdot 4 \text{ B} = 201.3 \text{ MB}$ per layer, far larger than the on-chip caches; the dense median of 4.01–4.13 ms (EXP-005) corresponds to an effective 49–50 GB/s. The TinyLlama FFN (138.4 MB) ran in 2.64–2.69 ms (EXP-017), or 51–52 GB/s. Both are about half of M2’s specified peak bandwidth and roughly constant across geometries. These are derived estimates; we did not measure memory-system counters.

Suppose a sparse executor streams its active weights at a fraction ρ of the dense rate. Then $t_{\text{sparse}}(s) = (1 - s) t_{\text{dense}}/\rho$ and

$$\text{speedup}(s) = \frac{\rho}{1 - s} - 1, \quad s^* = 1 - \rho, \quad (3)$$

where s^* is the break-even sparsity. Inverting the measurements, $\rho = (1 - s)(1 + \text{speedup})$, gives a direct efficiency estimate at every grid point.

Packed B8. ρ is 0.65 at 10% falling to 0.57 at 80% for Llama, and 0.61 at 20% falling to 0.56 at 60% for TinyLlama. The lowest-sparsity values predict break-even near 35% and 39%, inside the measured Llama interval (30%, 40%) and just below the TinyLlama interval (40%, 50%]. Because ρ falls with sparsity, these predictions are lower bounds rather than precise estimates.

Per-neuron. On the same masks, ρ is 0.92–0.97 at Llama B8/10% (0.91–0.93 on real masks) and 0.84–0.86 at TinyLlama B8/20%. The per-neuron executor is thus nearly as efficient as vendor GEMV per byte.

6.2 Why the packed layout is slow

The two sparse executors perform identical arithmetic but traverse the down projection differently. The packed $[M, h, B]$ layout turns each active block into $h = 2048$ dot products of length 8, each updating one output element, whereas the neuron-major layout turns each active neuron into one long, contiguous scaled-row addition that vectorizes well. Profiling (EXP-009) attributes 59% of packed-B8 time to the down projection. Explicit NEON intrinsics for the short dot products were slower still (EXP-010). This explanation is consistent with the measurements but is an interpretation; we did not collect hardware counters.

6.3 The ceiling on any kernel

Equation 3 also bounds what any kernel can achieve. Even a perfect executor ($\rho = 1$) at the quality ceiling s_q saves only s_q of the bytes, a speedup of $1/(1 - s_q) - 1$: 11.1% for Llama ($s_q = 10\%$) and 25% for TinyLlama ($s_q = 20\%$). The per-neuron executor already captures 2.6–7.3 of Llama’s possible 11.1 points and 5.1–7.3 of TinyLlama’s 25. The FFN is only part of token latency, so the end-to-end gain would be smaller still, and a deployable selector must cost less than this margin. Larger gains therefore require moving the *quality* ceiling, not the kernel.

6.4 Supporting experiments

Table 3 summarizes the thirteen experiments that do not enter the headline frontier but constrain its interpretation. Three findings matter most. First, naively expanding neuron-level masks to blocks is not viable: at B8, 40% neuron sparsity left a mean of only 0.066% block sparsity because almost every block contains at least one selected neuron, and a random neuron permutation gave the same result, so model neuron order carries no useful contiguity. Second, cheap reorderings fitted on four calibration prompts help on held-out prompts but not enough: a hot/cold layout reduced Llama B8/20% perplexity increase from +8.77% to +6.80%, still above the gate. Third, alternative packed-B8 implementations were slower: explicit NEON by 3.6–6.7% and coalescing contiguous active runs into BLAS calls by 28–30%. Quantized dense execution raises the bar further: a matched llama.cpp [4] Q8_0 (8-bit) dense FFN with the Llama geometry ran in 1.3–1.5 ms, and a minimal Q8_0 B32 group-skipping path did not reliably beat it even at 80% sparsity.

7 Discussion

What the evidence supports. On equal-work block-aligned masks, packed B8 execution overtakes Accelerate dense only at 40–50% sparsity, well beyond what the tested models tolerate. The same masks executed neuron-by-neuron with neuron-major weights are faster

than dense at the quality ceiling, by a few percent. Block alignment is therefore useful on this CPU, but its value comes from keeping active weights in long contiguous runs, not from packing blocks into short tiles. The quality ceiling, not kernel efficiency, limits the achievable gain: at 10–20% sparsity even a perfect kernel could save only 11–25% of FFN time.

Where the bottleneck is. The structure tax in Section 5.1—a 30% budget costs 0.2–0.3% perplexity at neuron granularity but 6.9–20.2% at B8—shows that the missing ingredient is a *representation* in which useful neurons are contiguous. Post-hoc reordering moved the quality frontier only slightly. More plausible routes are training-time or fine-tuning methods that make activations block-sparse by construction, such as natively trained mixture-of-experts (MoE) FFNs, ReLU-restoring fine-tuning [13], or joint selector and layout learning. Any such continuation should be compared against quantized dense baselines, which are already roughly three times faster than FP32 here.

On reporting negative results. Fixing the quality gate and grids in version control before running each experiment [15] prevented the gate from being loosened once results were known, and retaining every completed experiment, including failed variants, in the artifact lets readers verify that the headline result is not the product of selective reporting. The per-neuron finding itself was already present in the recorded data; it became visible only when every executor in the artifact was compared against the quality frontier, which is an argument for releasing all raw results.

7.1 Threats to validity

- **Small quality screen.** 16 excerpts and 2,032 tokens, perplexity only, no downstream tasks. Exact ceilings may shift with a larger corpus.
- **Oracle masks.** Selection reads dense activations that a deployable predictor cannot see, so real predictors are expected to do worse. The score ranks blocks independently, however, so a better subset could exist; we do not claim the oracle is optimal.
- **Synthetic runtime masks and weights.** Break-even sweeps use random block-aligned masks and random weights. At Llama B8/10% they agreed with real masks for packed B8; real masks gave a smaller per-neuron gain (0.9–3.5% versus 2.6–7.3%). This was not checked at other sparsities or for TinyLlama.
- **Uncontrolled system state.** macOS does not allow pinning a thread to a performance core, and we did not control frequency, thermal state, or power source. Dense medians varied by up to 17% across EXP-012 cases. Randomized executor order limits bias between executors but not run-to-run variation.

Table 3: Supporting experiments. All use the same hardware, provenance checks, and validators as the decisive experiments. Speedups use the paper’s metric $t_{\text{dense}}/t_{\text{sparse}} - 1$.

ID	Question	Observation
EXP-001	Do blocks beat per-neuron execution at equal work (synthetic)?	Some block configurations were faster, but none beat per-neuron execution in every run while computing extra neurons. Per-neuron execution on scattered masks was usually slower than dense.
EXP-002	Can real neuron masks be expanded to blocks?	No: B8 kept 0.0002–0.066% block sparsity at 20–40% neuron sparsity; B32/B64 kept exactly 0%. A random permutation gave the same result.
EXP-003	Does activation-sum block scoring preserve quality?	Neuron top- k 30%: +0.21%. B8–B64 at 30%: +21.3% to +78.4%.
EXP-006	Does a locality-sensitive-hashing (LSH) co-activation layout help?	Improved every matched B8/B16 point (e.g. B8/20% +8.56%) but none passed.
EXP-007/008	Does hot/cold neuron ordering (fit on 4 prompts) help?	Best layout tested (B8/20% +6.80%, B8/30% +15.58%), still failing.
EXP-009	Where does packed-B8 time go?	Down projection 59%, gate/up/SwiGLU 41%, mask scan 0.02%. Staged execution was 7.9–10.5% slower than interleaved.
EXP-010	Does explicit NEON help packed B8?	3.6–6.7% slower than the auto-vectorized kernel.
EXP-011	Does coalescing active runs into BLAS calls help?	28–30% slower (median 93 runs, ≈ 279 small calls per FFN).
EXP-013	Does a custom INT8 kernel move the break-even?	B8 beats matched custom INT8 dense from 10%, but that dense kernel (6.8–7.3ms) is far slower than FP32 Accelerate (≈ 4 ms).
EXP-014	Strong quantized dense baseline?	llama.cpp Q8_0, CPU only: 125 tok/s prefill, 40 tok/s generation.
EXP-015	Does Q8_0-aligned B32 skipping help?	No reproducible speedup at 10–80%; –58% to –57% at 10%.
EXP-017	Is TinyLlama B8/20% fast with packed B8?	FP32: –24.5% to –20.2% (a separate run from EXP-018). Custom INT8 B8 beats matched INT8 dense but not FP32 dense.

- **Narrow hardware scope.** One Apple M2, one thread, one token, FP32 kernels, two FFN geometries. Multi-threaded, batched, or GPU/NPU execution may behave differently.
- **Statistics.** Reported ranges are across three repeated runs, not confidence intervals. The smallest positive result (0.9% on real masks) is within plausible run-to-run noise even though it was positive in every run.

7.2 Reproducibility

The artifact contains source code, pinned model revisions, raw per-sample timings, per-record log-likelihoods, manifests with hashes, independent validators, and the scripts that regenerate the main figures and tables in this paper. The committed result package can be validated without downloading either model.

8 Conclusion

We asked whether contiguous FFN blocks can turn dynamic sparsity into practical CPU speedups on an Apple M2 without unacceptable quality loss. Llama 3.2 1B

and TinyLlama 1.1B tolerate only 10–20% B8 block sparsity under a 5% perplexity gate. A packed block executor needs 40–50% sparsity to beat dense, so it never reaches a useful operating point, but a per-neuron executor with neuron-major weights runs the same block-aligned masks faster than dense at the quality ceiling, by a few percent. On this hardware, block alignment helps through memory layout rather than packed kernels, and the quality ceiling caps the possible FFN gain at 11–25%. The main obstacle to larger gains is block-compatible model representation. We release the complete evidence as a baseline for methods that train block-compatible structure.

References

- [1] Keivan Alizadeh et al. LLM in a flash: Efficient large language model inference with limited memory. In *Proceedings of ACL*, 2024.
- [2] Marco Federici, Davide Belli, Mart van Baalen, Amir Jalalirad, Andrii Skliar, Bence Major, Markus Nagel, and Paul Whatmough. Efficient LLM inference using dynamic input pruning and cache-aware masking. In *Proceedings of Machine Learning and Systems*, 2025.

- [3] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [4] Georgi Gerganov et al. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2026. Commit fa6769818708afd9807b22183ccda112fd563427.
- [5] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [6] Scott Gray, Alec Radford, and Diederik P. Kingma. GPU kernels for block-sparse weights. Technical report, OpenAI, 2017.
- [7] Torsten Hoefer, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241):1–124, 2021.
- [8] Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-aware thresholding for sparsity in large language models. In *Conference on Language Modeling (COLM)*, 2024.
- [9] Zonglin Li, Chong You, Srinadh Bhojanapalli, Daliang Li, Ankit Singh Rawat, Sashank J. Reddi, Ke Ye, Felix Chern, Felix Yu, Ruiqi Guo, and Sanjiv Kumar. The lazy neuron phenomenon: On emergence of activation sparsity in transformers. In *International Conference on Learning Representations*, 2023.
- [10] James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-free activation sparsity in large language models. In *International Conference on Learning Representations*, 2025.
- [11] Zichang Liu et al. Deja Vu: Contextual sparsity for efficient LLMs at inference time. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [12] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.
- [13] Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU strikes back: Exploiting activation sparsity in large language models. In *International Conference on Learning Representations*, 2024.
- [14] Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378*, 2021.
- [15] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.
- [16] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [17] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. PowerInfer: Fast large language model serving with a consumer-grade GPU. In *Proceedings of the 30th Symposium on Operating Systems Principles*, 2024.
- [18] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations*, 2024.
- [19] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [20] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. PowerInfer-2: Fast large language model inference on a smartphone. *arXiv preprint arXiv:2406.06282*, 2024.
- [21] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.
- [22] Zhengyan Zhang, Yankai Lin, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. MoEfication: Transformer feed-forward layers are mixtures of experts. In *Findings of the Association for Computational Linguistics: ACL 2022*, 2022.